

PROJET DE FIN DE CYCLE (PFC)

SYSTÈME D'ALERTE MÉDICALE CONNECTÉ

Hospital Patient Alarm System

Rapport de documentation technique approfondie décrivant la conception matérielle,
l'architecture logicielle réseau (Node.js & Aedes MQTT), la structure de données
PostgreSQL (Drizzle ORM) et l'intégration mobile via Capacitor WebView.

Auteur :	Zinedine Aouda
Rôle :	Ingénieur Concepteur de Système IoT & Logiciel
Base de données :	PostgreSQL + Drizzle ORM
Microcontrôleurs :	ESP32-S3 (Contrôleur) & ESP8266 (Terminaux)
Date :	Mai 2026
Version :	2.0.0 (Cloud Monolith Migration)

1. Introduction Générale

Le présent projet consiste en la conception et la réalisation d'un **système d'alerte médicale connecté (Hospital Patient Alarm System)**. Ce dispositif a pour objectif principal d'optimiser la réactivité et la gestion des appels d'urgence au sein des établissements de santé. Il permet aux patients hospitalisés d'émettre des alertes instantanées depuis leur lit vers un poste de surveillance centralisé ou directement sur les terminaux mobiles du personnel soignant (infirmiers de garde).

1.1 Problématique

Dans la configuration hospitalière traditionnelle, les systèmes d'appel d'urgence physiques souffrent souvent de plusieurs limites critiques :

- **Coût et rigidité matérielle** : L'implémentation de systèmes entièrement filaires nécessite des travaux de câblage importants, ce qui augmente considérablement les coûts d'installation et de maintenance.
- **Perte de mobilité des infirmiers** : Les signaux d'alertes traditionnels sont souvent limités à des voyants lumineux de porte ou des moniteurs fixes dans la salle de garde. Les infirmiers en déplacement dans le service n'ont aucun moyen d'être avertis en temps réel à moins de retourner au poste central.
- **Vulnérabilité face aux coupures de réseau** : Les solutions cloud pures exposent le système à une panne critique si la connexion Internet externe est perdue. Dans un cadre de santé, une interruption de service de quelques minutes peut avoir des conséquences vitales.

1.2 Solution Proposée

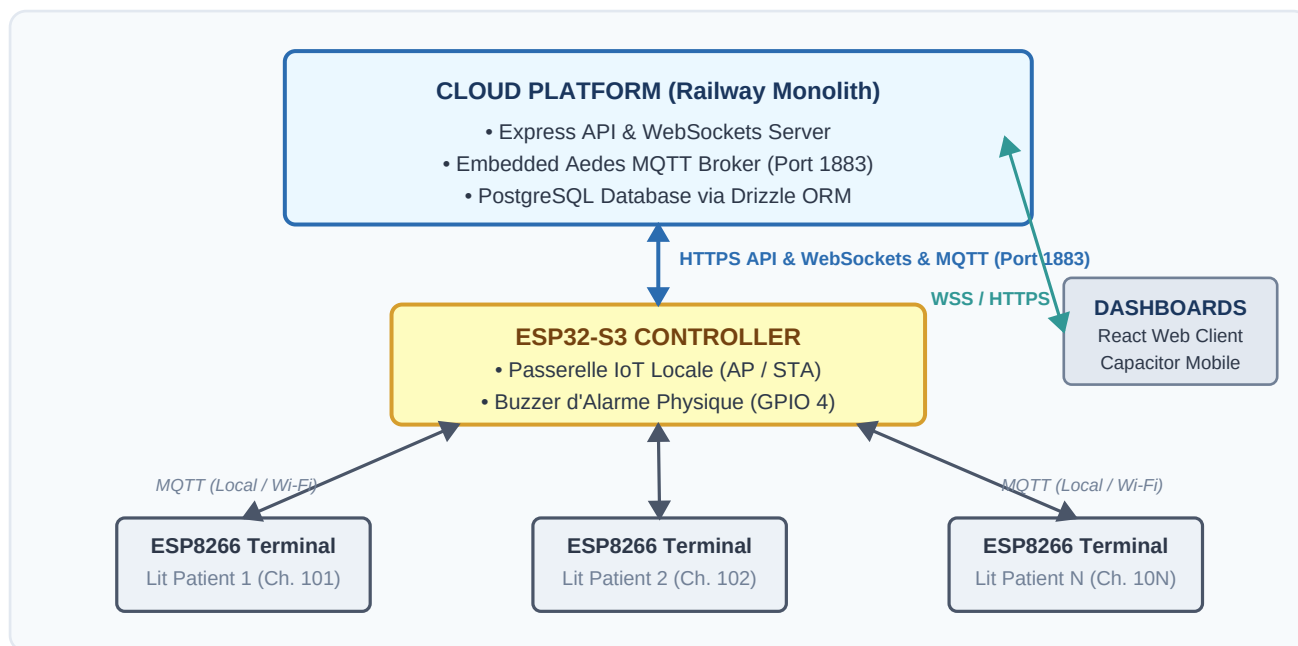
Pour résoudre ces enjeux majeurs de sécurité, de coût et de mobilité, nous avons développé une architecture connectée hybride et résiliente, reposant sur trois couches coordonnées :

1. **Une couche matérielle IoT résiliente** : Chaque chambre ou lit de patient dispose d'un boîtier émetteur **ESP8266** équipé d'un bouton poussoir. Un contrôleur central intelligent **ESP32-S3**, agissant comme passerelle locale, supervise les boîtiers de sa zone et pilote un buzzer d'alarme physique, garantissant le fonctionnement du système localement, même sans aucune liaison Internet.
2. **Une plateforme Cloud Centralisée (Railway Monolith)** : Un serveur Node.js / Express fait office de concentrateur global. Il embarque nativement son propre courtier (broker) **Aedes MQTT** et fournit les flux **WebSockets** temps réel tout en gérant la persistance dans une base de données **PostgreSQL**.
3. **Une application Mobile de Réception (Capacitor)** : L'application utilisée par le personnel soignant est bâtie en React encapsulée dans une WebView native de type **Capacitor**. Elle reçoit les notifications et alertes en temps réel, bénéficie des vibrations matérielles du téléphone, et offre une flexibilité de mise à jour instantanée sans recompilation d'APK.

2. Architecture Globale du Système

Récemment migré d'une infrastructure éclatée (Vercel Serverless pour l'API, HiveMQ externe pour le MQTT) vers une architecture **Cloud Monolithique centralisée**, le système offre désormais des gains majeurs en performance et en synchronisation. La centralisation du broker MQTT et du serveur de WebSockets au sein du même processus Node.js sur Railway permet de court-circuiter les latences réseaux.

Figure 2.1 : Topologie réseau et flux de données de l'architecture hybride



2.1 Les Terminaux Patients (ESP8266)

Chaque boîtier patient est conçu autour du circuit Wi-Fi low-cost ESP8266. Configuré pour consommer un minimum de courant au repos, il maintient une liaison MQTT ouverte avec le broker cloud. L'ESP8266 surveille une broche d'interruption physique reliée au bouton de lit. Lors de l'activation, il bascule sa LED interne à l'état fixe et transmet la commande.

2.2 Le Contrôleur Local (ESP32-S3)

L'ESP32-S3 sert de passerelle d'agrégation matérielle locale. Il établit une double interface : il peut se comporter comme un point d'accès Wi-Fi (Mode Access Point) pour les terminaux en l'absence de réseau hôte hospitalier, ou se connecter au Wi-Fi de l'établissement (Mode Station) pour synchroniser l'ensemble des données locales avec la plateforme Railway. En cas d'alerte, c'est lui qui déclenche la sirène d'alerte locale branchée sur sa sortie PWM/GPIO.

2.3 Le Serveur Cloud Monolithique (Node.js & Aedes)

Hébergée sur Railway, l'instance Node.js unifie le routage HTTP, les canaux de communication IoT et la distribution client. Le protocole MQTT transite par la bibliothèque **Aedes** configurée en écoute directe sur le port standard 1883. Lorsqu'un message d'alerte arrive, le broker l'intercepte, l'insère dans la base PostgreSQL via Drizzle ORM, et le transmet simultanément au serveur WebSocket (WSS) interne pour notifier instantanément les navigateurs et terminaux mobiles ouverts.

2.4 Le Dashboard Mobile (React & Capacitor)

Plutôt que d'employer une structure React Native complexe nécessitant des compilations régulières pour chaque correction d'interface, le dashboard mobile repose sur **Capacitor**. Cette technologie enveloppe la Single Page Application (SPA) React hébergée sur Railway. Le terminal mobile charge dynamiquement la page cloud tout en gardant accès aux APIs matérielles (vibreur, son de notification).

3. Spécifications du Matériel (Hardware)

La conception matérielle a été pensée pour être la plus épurée possible afin de maximiser la fiabilité et de réduire les pannes physiques. L'utilisation de résistances de pull-up internes sur les microcontrôleurs permet d'éliminer les résistances externes dans le câblage des boutons poussoirs.

3.1 Schéma des Connexions & Broches (Pin Mapping)

A. Contrôleur Central (ESP32-S3 DevKitC-1)

L'ESP32-S3 pilote l'avertisseur sonore d'alerte physique. Un buzzer actif est utilisé pour générer un son continu sans nécessiter de modulation de fréquence complexe dans le code.

Broche ESP32-S3	Composant	Rôle	Description / Détail technique
GPIO 4	Buzzer Actif (+)	Sortie Numérique	Activé à l'état HAUT (HIGH, 3.3V) pour déclencher l'alarme sonore physique locale.
GND	Buzzer Actif (-)	Masse (0V)	Masse de référence électrique commune.
USB-C	Port Série / Alim	Alimentation & Debug	Interface d'alimentation USB 5V et liaison de débogage UART à 115200 Bauds.

B. Terminal Bouton Patient (ESP8266 NodeMCU)

Chaque boîtier patient possède un unique bouton d'appel d'urgence et utilise sa LED intégrée (souvent située sur le module ESP-12F) pour confirmer l'état de l'appel au patient hospitalisé.

Broche ESP8266	Composant	Mode Configuration	Description / Détail technique
GPIO 0 (D3)	Bouton Poussoir	INPUT_PULLUP	Détecte l'appui. Le bouton relie la broche à la masse (GND). Actif à l'état BAS (LOW).
GPIO 2 (D4)	LED intégrée	OUTPUT	LED d'indication. Active à l'état BAS (LOW, 0V) et éteinte à l'état HAUT (HIGH, 3.3V).

3.2 Comportement Lumineux du Terminal Patient

Afin de fournir un retour d'expérience clair et sécurisant aux patients (souvent stressés ou âgés) et aux techniciens chargés de la maintenance, la LED du boîtier d'appel ESP8266 possède quatre états distincts :

- **Clignotement ultra-rapide (intervalle de 200ms)** : L'appareil est en cours de démarrage, d'établissement de la liaison Wi-Fi, ou de négociation avec le courtier MQTT Cloud. Indique une phase transitoire.
- **Clignotement lent (intervalle de 500ms)** : Le terminal a réussi à s'associer au réseau Wi-Fi local, mais il n'a pas encore reçu l'approbation administrative de l'hôpital. Il attend qu'un administrateur lui attribue une chambre et un lit dans la base de données.
- **Allumée en continu (Fixe H24)** : Une alerte est active. Le patient a pressé le bouton, le signal a été transmis et persisté. La LED reste allumée jusqu'à ce qu'une infirmière valide/acquitte l'alerte sur son écran.
- **Éteinte** : L'appareil est en ligne, dûment approuvé par l'administration, et aucun appel n'est actif. L'appareil est au repos et consomme le minimum d'énergie.

4. Protocoles de Communication

La communication inter-systèmes repose sur un modèle hybride associant légèreté pour l'IoT (MQTT) et réactivité instantanée pour l'humain (WebSockets WSS). L'Express API HTTP sert de support complémentaire pour les requêtes synchrones d'administration.

4.1 Protocole MQTT (Liaisons IoT)

MQTT (Message Queuing Telemetry Transport) est un protocole de messagerie de type publication/abonnement extrêmement léger, particulièrement adapté aux microcontrôleurs disposant de ressources CPU et réseau limitées.

A. Structure des Pings de Santé (Heartbeats) :

Chaque terminal ESP8266 et le contrôleur central publient périodiquement sur le topic :

`device/{deviceId}/heartbeat` ou `controller/{key}/ping`

Le payload JSON transmet la qualité du signal RSSI local et l'uptime matériel.

B. Canal d'Alerte :

Lors de l'appui sur le bouton, le terminal publie instantanément sur le topic :

`controller/{deviceKey}/alert`

avec un payload JSON identifiant le terminal. Le client MQTT interne au serveur Express intercepte ce message, met à jour l'état en base de données et redistribue l'alerte aux soignants.

4.2 API REST HTTPS

Bien que le temps réel soit géré par MQTT et WebSockets, les opérations administratives et d'administration courantes s'appuient sur des requêtes HTTP Express standards sécurisées :

- `POST /api/devices/approve` : Permet d'approuver un terminal en attente et de l'associer à une chambre et un lit.
- `POST /api/devices/clear-alert` : Déclenché par l'infirmière pour acquitter et éteindre une alerte.
- `POST /api/setup/mode` : Permet de configurer à distance le mode Wi-Fi du contrôleur central.

4.3 Protocole WebSocket (Mise à Jour Instantanée)

Afin de rafraîchir le dashboard des infirmiers sans générer de requêtes de polling HTTP incessantes, une connexion **WebSockets sécurisée (WSS)** permanente est maintenue entre le client React (Web/Capacitor) et le serveur Node.js. Les événements transmis sont les suivants :

- `FULL_STATE` : Transmis immédiatement après la connexion d'un client pour lui envoyer la liste complète des appareils et leur état.
- `ALERT` : Diffusé dès qu'une alerte est persistée en base de données, déclenchant l'alarme sonore et visuelle sur l'appareil de l'infirmier.
- `UPDATE / DELETE` : Notifie le dashboard de l'approbation d'un nouveau lit ou du retrait d'un matériel.
- `CONTROLLER_STATUS` : Permet de remonter la force Wi-Fi (RSSI) et l'état en ligne du contrôleur ESP32-S3 sur le panneau de supervision.

5. Modèle de Données & Structure Logicielle

La persistance des données repose sur **PostgreSQL** hébergé sur Railway, requêté via l'ORM TypeScript **Drizzle** et validé en amont grâce à des schémas **Zod** rigoureux.

5.1 Schémas des Tables Drizzle

Table 1 : devices

Nom Colonne	Type SQL	Défaut	Description / Usage applicatif
device_id	TEXT	PRIMARY KEY	Identifiant unique basé sur l'adresse MAC du boîtier (ex: device-2b8f41).
patient_name	TEXT	"	Nom complet du patient occupant le lit attribué.
bed	TEXT	"	Numéro ou libellé du lit dans la chambre.
room	TEXT	"	Numéro de la chambre ou de l'unité médicale.
alert_active	BOOLEAN	false	Indique si une alerte d'urgence est active et en cours.
last_alert_time	TEXT	NULL	Horodatage ISO de la dernière alerte déclenchée.
registered	BOOLEAN	false	true si l'appareil a contacté le serveur au moins une fois.
approved	BOOLEAN	false	true si l'administrateur a validé l'appareil et son affectation.
online	BOOLEAN	false	Indique l'état de connexion actif (heartbeats réguliers).
last_seen	BIGINT	NULL	Timestamp Unix (ms) du dernier heartbeat reçu.
last_updated_at	BIGINT	NULL	Timestamp Unix (ms) de la dernière modification d'état.

Table 2 : system_settings

Nom Colonne	Type SQL	Défaut	Description / Usage applicatif
id	INTEGER	PRIMARY KEY (1)	Clé primaire forcée à 1 pour assurer une configuration singleton unique.
controller_last_seen	BIGINT	NULL	Timestamp Unix de la dernière communication de l'ESP32-S3.
controller_uptime	INTEGER	0	Temps de fonctionnement cumulé de l'ESP32-S3 (en secondes).
controller_rssi	INTEGER	0	Force du signal Wi-Fi capté par le contrôleur (en dBm).
controller_wifi_error	TEXT	NULL	Dernier message d'erreur réseau renvoyé par l'ESP32-S3.
wifi_mode	INTEGER	1	Mode Wi-Fi configuré (1: AP, 2: STA, 3: AP+STA, 4: Cloud Sync).
pending_command	TEXT	NULL	Commande en attente de synchronisation pour le contrôleur.
command_params	TEXT	NULL	Paramètres JSON additionnels pour la commande en attente.

5.2 Structure du Projet (Monorepo)

Le projet est organisé sous forme de monorepo TypeScript propre, clarifiant la séparation entre le micrologiciel embarqué C++ et la logique d'application Web et Mobile :

```
My-PFC/
|-- server/ # Backend Node.js / Express / WebSockets & Aedes MQTT
| |-- index.ts # Point d'entrée de l'application & initialisation de la base
| |-- routes.ts # Définition des endpoints REST HTTP & Middleware
| |-- wss.ts # Gestion du serveur de WebSockets
| |-- mqtt.ts # Logique du broker MQTT intégré (Aedes) & gestion des queues
| |-- storage.ts # Logique d'accès aux données (PostgreSQL via Drizzle ORM)
| |-- db.ts # Configuration du pool de connexion PostgreSQL
| |-- static.ts # Serveur de fichiers statiques pour le SPA React
|-- shared/ # Code partagé entre le frontend et le backend
| |-- schema.ts # Modèles de base de données Drizzle et schémas de validation Zod
|-- client/ # Application Frontend React (Servie par Node.js)
| |-- src/
| | |-- App.tsx # Routage (wouter) et initialisation
| | |-- components/ # Composants réutilisables (DeviceCard, SetupWizard)
| | |-- hooks/ # Hooks personnalisés (useAlertSound, use-mobile)
| |-- vite.config.ts # Config de build frontend React
|-- android/ # Application Mobile (Capacitor WebView)
| |-- capacitor.config.ts # Pointe l'application mobile vers https://my-pfc-production.up.railway.app
|-- esp32/ # Firmware des microcontrôleurs (C++)
|-- controller/ # Code du contrôleur central ESP32-S3 (Buzzer, Passerelle)
|-- device/ # Code du boîtier patient ESP8266 (Bouton, LED)
```

6. Sécurité, Avantages & Résilience

La conception technique de la nouvelle architecture monolithique centralisée offre des garanties robustes en termes de résilience matérielle, de réactivité réseau et de facilité de déploiement en production.

6.1 Avantages de l'Intégration Mobile via Capacitor

L'adoption de **Capacitor WebView** en remplacement d'une architecture native compilée (type React Native classique) présente des bénéfices considérables pour l'exploitation hospitalière :

- **Mises à jour instantanées (Over-The-Air)** : Toute correction de bug ou amélioration du tableau de bord effectuée sur le dépôt git et déployée sur Railway est instantanément visible sur le smartphone des infirmiers. Il n'est pas nécessaire de reconstruire le fichier APK, ni de forcer les soignants à réinstaller manuellement l'application.
- **Coquille native optimisée** : Capacitor permet de conserver l'accès aux vibrations physiques, aux notifications push système et à l'état de la batterie du smartphone, tout en bénéficiant de la simplicité et de la rapidité d'exécution du HTML5 / React moderne.

6.2 Robustesse face aux limitations Serverless

Initialement hébergé sur Vercel, le backend rencontrait des limites inhérentes aux fonctions Serverless : déconnexions incessantes des WebSockets dues aux temps d'exécution maximum (timeouts de 10 à 60 secondes) et incapacité de faire tourner un broker MQTT persistant en arrière-plan. L'hébergement sous forme de **Monolithe Railway (Dockerisé)** garantit un processus Node.js persistant H24. La latence d'acheminement d'une alerte est inférieure à 100ms, et la charge CPU du serveur reste minime.

6.3 File d'Attente de Commandes (Resilience Mode)

Le module de communication MQTT du serveur (`server/mqtt.ts`) intègre une gestion intelligente des files d'attente hors-ligne. Si l'ESP32-S3 perd momentanément sa connexion Wi-Fi à cause d'une perturbation dans l'hôpital, les requêtes vitales (comme l'acquiescement ou l'arrêt de la sirène) ne sont pas perdues. Elles sont empilées dans une queue en mémoire et re-émises immédiatement dès la détection de la reconnexion de la passerelle.

6.4 Modèle de Sécurité Hybride

Les données transitant dans le réseau hospitalier sont protégées par une authentification à deux niveaux :

1. **Sécurité IoT** : Les microcontrôleurs (ESP32 / ESP8266) s'authentifient auprès du broker MQTT cloud via des identifiants sécurisés et injectent un token d'appareil unique (`X-Device-Key`) dans l'en-tête de toutes les communications REST HTTP, évitant l'usurpation de lit.
2. **Sécurité Staff Médical** : L'accès au panneau de configuration et de suivi web par les infirmières et médecins est restreint par un mécanisme d'authentification classique (`Passport.js`) reposant sur des sessions chiffrées par cookie sécurisé.